CODING WALKTHROUGH

The Transformer in Code: Start to End

How a translation Transformer is built, trained and used, one step of code at a time, with the real output of every step.

Two programs, both run for this guide

Part A a complete encoder-decoder Transformer written from scratch in PyTorch (176,896 parameters), trained with teacher forcing on an English-Kannada phrasebook.

Part B the same flow with a real pretrained model, NLLB-200, using Hugging Face Transformers.

Every code listing is the real file from the repository (with its line numbers), and every output block was copied from an actual run.

Files: code/transformer_walkthrough/scratch_transformer.py and nllb_step_by_step.py · logs in code/results/ · Python 3.11, PyTorch 2.14, Transformers 5.20

How to run it, and the map of the code

Setup and commands

terminal

```
# one-time: create an environment and install the libraries
python -m venv .venv
.venv\Scripts\activate          # Linux/macOS: source .venv/bin/activate
pip install torch transformers

# Part A: from scratch (about 50 seconds on a laptop CPU, no GPU needed)
python code/transformer_walkthrough/scratch_transformer.py

# Part B: real pretrained model (downloads about 2.4 GB the first time)
python code/transformer_walkthrough/nllb_step_by_step.py
```

The pipeline and where each step lives in the code

#	Step	Part A: from scratch	Part B: Hugging Face
1	Collect aligned pairs	`json.load(corpus.json)`	(the model was already trained on large data)
2	Tokenize, split, vocabulary	`tokenize`, `build_vocab`, `encode_src/tgt`	`AutoTokenizer`
3	Batches, padding, masks	`make_batch`, `src_mask_of`, `tgt_mask_of`	`attention_mask`, `padding=True`
4	Embeddings + position	`PositionalEncoding`, `nn.Embedding`	inside the model
5-6	Attention	`attention`, `MultiHeadAttention`	`output_attentions=True`; head check by hand
7-8	Encoder and decoder layers	`EncoderLayer`, `DecoderLayer`	`get_encoder()`, `get_decoder()`
9-10	Whole model, shape trace	`Transformer.encode / decode / forward`	`memory = encoder(...)`
11-12	Loss and training	`run_epoch`, `F.cross_entropy`, `AdamW`	(pretrained; not trained here)
13	Decode	`translate` (greedy loop)	manual loop, `generate()`, beam search
14	Evaluate	`corpus_bleu`	(see the NMT page for BLEU on NLLB)

THE DATA FLOW IN ONE LINE

text → token ids → (batch, padded) → **embeddings + position** → **encoder layers** → memory → **decoder layers** (masked self-attention, cross-attention to the memory) → logits over the Kannada vocabulary → softmax → loss while training, or argmax and loop while translating.

ABOUT THE NUMBERS

Part A trains on just 50 sentence pairs and tests on 10, so its scores show that the code *works*, not how good a production translator would be. Part B is a real 615-million-parameter model.

PART A · TRANSFORMER FROM SCRATCH (PYTORCH)

1. Collect the aligned sentence pairs

A translation model learns from pairs: an English sentence and its Kannada translation. The pairs live in one JSON list, so index *i* keeps the source and its translation together; never sort or shuffle the two sides separately.

scratch_transformer.py

```
15  pairs = json.load(open("study_guide/nmt/corpus.json", encoding="utf-8"))          # [{"en": ..., "kn": ...}, ...]
16  print("STEP 1  pairs:", len(pairs), "| example:", pairs[0]["en"], "->", pairs[0]["kn"])
17
```

Output you get (real run)

```
STEP 1  pairs: 70 | example: Where is the bus stand? -> ಬಸ್ ನಿಲ್ದಾಣ ಎಲ್ಲಿದೆ?
```

Reading the code

- **Line 15:** `encoding='utf-8'` matters: Kannada text must be read as UTF-8 or it becomes garbage.
- **Line 16:** Always print a sample pair first: it proves the two sides are aligned.

WATCH OUT

If sources and targets get out of order (for example sorting only one list), the model silently learns nonsense and the loss still goes down.

2. Tokenize, split and build the vocabulary

Text becomes numbers in three moves. (1) **Tokenize**: cut a sentence into tokens. (2) **Split** the pairs into train, validation and test. (3) **Build the vocabulary from the training pairs only** and map every token to an integer id; unseen words become `<unk>`. The encoders add `</s>` to the source and `<s>` … `</s>` around the target.

scratch_transformer.py

```
19  def tokenize(text):
20      """Words, with ? . , ! split off as their own tokens (works for Kannada: no letter classes needed)."""
21      return re.findall(r"[^\s?.,!]+|[?.,!]", text)
22
23  # train / validation / test split (70 / 15 / 15), reproducible; every word of val/test must also occur in train
24  rng = random.Random(7)
25  order = list(range(len(pairs))); rng.shuffle(order)
26  words = lambda i: {("e", w.lower()) for w in tokenize(pairs[i]["en"])} | {("k", w) for w in tokenize(pairs[i]["kn"])}
27  count = {}
28  for i in order:
29      for w in words(i): count[w] = count.get(w, 0) + 1
30  n_val = n_test = round(0.15 * len(pairs)); val, test = [], []
31  for i in order:
32      if len(val) >= n_val and len(test) >= n_test: break
33      if all(count[w] > 1 for w in words(i)):
34          for w in words(i): count[w] -= 1
35          (test if len(test) < n_test else val).append(i)
36  held = set(val) | set(test); train = [i for i in order if i not in held]
37  print(f"STEP 2  split: train {len(train)} | validation {len(val)} | test {len(test)}")
38
39  def build_vocab(sentences, lower):
40      counts = {}
41      for s in sentences:
42          for w in tokenize(s.lower() if lower else s): counts[w] = counts.get(w, 0) + 1
43      itos = ["<pad>", "<s>", "</s>", "<unk>"] + sorted(counts, key=lambda w: (-counts[w], w))
44      return itos, {w: i for i, w in enumerate(itos)}          # vocabulary comes from TRAINING data only
45
46  src_itos, src_stoi = build_vocab([pairs[i]["en"] for i in train], True)
47  tgt_itos, tgt_stoi = build_vocab([pairs[i]["kn"] for i in train], False)
48  encode_src = lambda s: [src_stoi.get(w, UNK) for w in tokenize(s.lower())] + [EOS]              # x_1 ... x_S </s>
49  encode_tgt = lambda s: [BOS] + [tgt_stoi.get(w, UNK) for w in tokenize(s)] + [EOS]              # <s> y_1 ... y_T </s>
50  print("        vocab sizes:", len(src_itos), "(English),", len(tgt_itos), "(Kannada)")
51  ex = pairs[train[0]]
52  print("        example ids:", encode_src(ex["en"]), "|", encode_tgt(ex["kn"]))
53
```

Output you get (real run)

```
STEP 2  split: train 50 | validation 10 | test 10
        vocab sizes: 74 (English), 70 (Kannada)
        example ids: [68, 73, 70, 22, 51, 45, 54, 4, 2] | [1, 33, 66, 50, 52, 4, 2]
```

Reading the code

- **Line 19:** A regular expression: words, with `? . , !` split off as separate tokens. It works for Kannada because it never relies on letter classes.
- **Line 25:** A seeded shuffle: the split is reproducible.
- **Line 33:** Coverage rule: a pair may go to validation/test only if every word still occurs in training (this tiny corpus has no subword tokenizer).
- **Line 43:** Index 0-3 are reserved: pad, start, end, unknown. The rest are sorted by frequency.
- **Line 49:** Target = `<s>` + tokens + `</s>`. The start token is what the decoder reads first; the end token is what it must learn to predict.

WATCH OUT

Building the vocabulary on all data (including test) leaks information. Forgetting the end token means the model never learns when to stop.

3. Batches, padding and masks

Sentences differ in length, so a batch is padded with a pad id to one rectangle. Two masks tell attention what it may look at: the **source mask** hides the padding, and the **target mask** hides padding *and the future* (a lower-triangular matrix). The **teacher-forcing shift** also happens here: the decoder input is the target without its last token, the training target is the target without its first.

scratch_transformer.py

```
55  def make_batch(idx):
56      S = [encode_src(pairs[i]["en"]) for i in idx]; T = [encode_tgt(pairs[i]["kn"]) for i in idx]
57      pad = lambda seqs: torch.tensor([s + [PAD] * (max(map(len, seqs)) - len(s)) for s in seqs])
58      src, tgt = pad(S), pad(T)
59      return src, tgt[:, :-1], tgt[:, 1:]                    # decoder input = tgt without last, target = tgt shifted
    left
60
61  def src_mask_of(src):                                      # (B,1,1,S) True where the key is a real token
62      return (src != PAD)[:, None, None, :]
63
64  def tgt_mask_of(tgt_in):                                   # (B,1,T,T) real token AND not in the future
65      T = tgt_in.size(1)
66      causal = torch.tril(torch.ones(T, T, dtype=torch.bool, device=tgt_in.device))
67      return causal[None, None] & (tgt_in != PAD)[:, None, None, :]
68
69  src, tgt_in, tgt_out = make_batch(train[:3])
70  print("STEP 3  src", tuple(src.shape), "tgt_in", tuple(tgt_in.shape), "tgt_out", tuple(tgt_out.shape))
71  print("        causal mask for T=4:\n", tgt_mask_of(torch.ones(1, 4, dtype=torch.long))[0, 0].int())
72
```

Output you get (real run)

```
STEP 3  src (3, 9) tgt_in (3, 7) tgt_out (3, 7)
        causal mask for T=4:
 tensor([[1, 0, 0, 0],
        [1, 1, 0, 0],
        [1, 1, 1, 0],
        [1, 1, 1, 1]], dtype=torch.int32)
```

Reading the code

- **Line 57:** Pad every sequence to the longest in the batch.
- **Line 59:** The shift: input `<s> y1 y2`, target `y1 y2 </s>`. At position *t* the decoder sees the words before *t* and must predict word *t*.
- **Line 62:** Shape (batch, 1, 1, S) so it broadcasts over heads and query positions. True means *may attend*.
- **Line 66:** Lower triangle of ones: row *i* may look at columns ≤ *i*. Printed below as the matrix of 1s and 0s.

WATCH OUT

Mask polarity is the classic bug: here True = allowed, and the attention code uses `~mask` to block. Mixing the two conventions makes the model attend only to padding.

4. Embeddings and positional encoding

An `nn.Embedding` turns each token id into a learned vector of *d* numbers. Attention looks at all words at once, so order must be added by hand: a fixed sine and cosine pattern is added to the embedding of each position. The class below builds that table once and stores it as a buffer (saved with the model, never trained).

scratch_transformer.py

```
74  class PositionalEncoding(nn.Module):
75      def __init__(self, d, max_len=64):
76          super().__init__()
77          pos = torch.arange(max_len)[:, None]; k = torch.arange(0, d, 2)[None]
78          angle = pos / (10000 ** (k / d))                    # pos / 10000^(2k/d)
79          pe = torch.zeros(max_len, d); pe[:, 0::2] = torch.sin(angle); pe[:, 1::2] = torch.cos(angle)
80          self.register_buffer("pe", pe)                      # fixed, not learned
81      def forward(self, x):                                   # x: (B, L, d)
82          return x + self.pe[: x.size(1)]
83
```

Reading the code

- **Line 78:** The formula pos / $10000^{2k/d}$: low dimensions oscillate fast, high dimensions slowly.
- **Line 79:** Even columns sine, odd columns cosine.
- **Line 80:** Part of the module's state but not a parameter: the optimiser never touches it.
- **Line 82:** Added, not concatenated, so the width stays *d*.

WATCH OUT

The model code multiplies embeddings by √d before adding position (see step 9); without that scale the position signal swamps small embeddings.

5. Scaled dot-product attention

The heart of the model in four lines: score every query against every key, divide by $\sqrt{d_k}$, block forbidden positions with $-\infty$, softmax into weights that sum to 1, and take the weighted average of the values. Shapes: queries (B, h, Tq, dk), keys and values (B, h, Tk, dk).

scratch_transformer.py

```
85  def attention(Q, K, V, mask=None):
86      """Q: (B,h,Tq,dk)  K,V: (B,h,Tk,dk)  mask: True = may attend, broadcastable to (B,h,Tq,Tk)."""
87      scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))  # (B,h,Tq,Tk)
88      if mask is not None:
89          scores = scores.masked_fill(~mask, float("-inf"))     # blocked positions get weight 0 after softmax
90      weights = F.softmax(scores, dim=-1)
91      return weights @ V, weights
92
```

Reading the code

- **Line 87:** $Q \cdot K^T$: one score per (query position, key position) pair, shape (B, h, Tq, Tk), divided by √dk.
- **Line 89:** Blocked positions get $-\infty$, so softmax gives them exactly 0 weight.
- **Line 90:** Softmax over the key axis: each query's weights sum to 1.
- **Line 91:** Weighted average of the values, shape (B, h, Tq, dk); the weights are returned too so they can be inspected.

WATCH OUT

A row whose keys are *all* masked gives NaN (softmax of all $-\infty$). The masks here always leave at least one real key per row.

6. Multi-head attention

One attention can only learn one kind of relationship, so the model runs *h* attentions in parallel on slices of the vector. Four linear layers do the work: `Wq, Wk, Wv` project the input, the result is split into heads, attended, joined, and mixed by `Wo`. The same class serves self-attention (`q_in` and `kv_in` are the same tensor) and cross-attention (queries from the decoder, keys and values from the encoder memory).

scratch_transformer.py

```
94  class MultiHeadAttention(nn.Module):
95      def __init__(self, d, heads):
96          super().__init__()
97          assert d % heads == 0
98          self.h, self.dk = heads, d // heads
99          self.Wq, self.Wk, self.Wv, self.Wo = (nn.Linear(d, d) for _ in range(4))
100     def split(self, x):                                  # (B,L,d) -> (B,h,L,dk)
101         B, L, _ = x.shape
102         return x.view(B, L, self.h, self.dk).transpose(1, 2)
103     def forward(self, q_in, kv_in, mask=None):
104         Q, K, V = self.split(self.Wq(q_in)), self.split(self.Wk(kv_in)), self.split(self.Wv(kv_in))
105         out, self.last_weights = attention(Q, K, V, mask)     # weights kept so we can inspect them
106         B, _, Tq, _ = out.shape
107         return self.Wo(out.transpose(1, 2).reshape(B, Tq, -1))   # join heads, mix with Wo
108
```

Reading the code

- **Line 97:** The width must divide evenly across heads: 64 / 4 = 16 numbers per head.
- **Line 102:** Reshape (B, L, d) to (B, L, h, dk), then move the head axis forward: (B, h, L, dk).
- **Line 104:** Queries come from `q_in`, keys and values from `kv_in`: this one line is the difference between self- and cross-attention.
- **Line 105:** Stored so the attention maps can be examined after a forward pass.
- **Line 107:** Heads are joined back to width *d* and mixed by `Wo`.

WATCH OUT

`.view` needs contiguous memory; after `transpose` use `.reshape` (as the code does when joining heads).

7 and 8. The encoder layer and the decoder layer

Each layer is attention plus a small feed-forward network, each wrapped in a skip connection (`x + sub_layer(x)`) and a LayerNorm. The **encoder layer** has one self-attention. The **decoder layer** has three sub-layers: masked self-attention (looks at the target so far), **cross-attention** (queries from the decoder, keys and values from the encoder memory) and a feed-forward network. LayerNorm comes before each sub-layer (pre-norm), as in the real NLLB model.

scratch_transformer.py

```
110 class FeedForward(nn.Module):
111     def __init__(self, d, ff, drop):
112         super().__init__(); self.net = nn.Sequential(nn.Linear(d, ff), nn.ReLU(), nn.Dropout(drop), nn.Linear(ff, d))
113     def forward(self, x): return self.net(x)
114
115 class EncoderLayer(nn.Module):                                  # pre-norm: LayerNorm before each sub-layer
116     def __init__(self, d, heads, ff, drop):
117         super().__init__()
118         self.attn, self.ffn = MultiHeadAttention(d, heads), FeedForward(d, ff, drop)
119         self.ln1, self.ln2, self.drop = nn.LayerNorm(d), nn.LayerNorm(d), nn.Dropout(drop)
120     def forward(self, x, src_mask):
121         h = self.ln1(x); x = x + self.drop(self.attn(h, h, src_mask))       # self-attention + skip
122         return x + self.drop(self.ffn(self.ln2(x)))                          # feed-forward + skip
123
124 class DecoderLayer(nn.Module):
125     def __init__(self, d, heads, ff, drop):
126         super().__init__()
127         self.self_attn, self.cross_attn, self.ffn = MultiHeadAttention(d, heads), MultiHeadAttention(d, heads),
    FeedForward(d, ff, drop)
128         self.ln1, self.ln2, self.ln3, self.drop = nn.LayerNorm(d), nn.LayerNorm(d), nn.LayerNorm(d), nn.Dropout(drop)
129     def forward(self, y, memory, tgt_mask, src_mask):
130         h = self.ln1(y); y = y + self.drop(self.self_attn(h, h, tgt_mask))              # masked self-attention
131         y = y + self.drop(self.cross_attn(self.ln2(y), memory, src_mask))               # Q: decoder, K,V: encoder
    memory
132         return y + self.drop(self.ffn(self.ln3(y)))
133
```

Reading the code

- **Line 121:** Self-attention: the same normalised tensor `h` is query, key and value.
- **Line 121:** The skip connection: the sub-layer output is *added* to its input, which keeps gradients flowing through deep stacks.
- **Line 130:** Masked self-attention: `tgt_mask` hides future words.
- **Line 131:** Cross-attention: query from the decoder state `y`, keys and values from `memory`; `src_mask` hides source padding.
- **Line 112:** Feed-forward: widen to `ff`, ReLU, narrow back to *d*; applied to each position alone.

WATCH OUT

Cross-attention must receive the *encoder's final output* (`memory`), the same tensor in every decoder layer, and the source mask, not the target mask.

9. Assemble the whole model

The model class wires the pieces together and exposes two separate methods, because they run at different times: `encode` runs **once** per source sentence and returns the memory; `decode` runs once per training batch, or once per generated token when translating. The output layer reuses (ties) the target embedding matrix, which saves parameters.

scratch_transformer.py

```
135 class Transformer(nn.Module):
136     def __init__(self, n_src, n_tgt, d=64, heads=4, layers=2, ff=128, drop=0.1):
137         super().__init__()
138         self.d = d
139         self.src_emb, self.tgt_emb = nn.Embedding(n_src, d, padding_idx=PAD), nn.Embedding(n_tgt, d, padding_idx=PAD)
140         self.pos, self.drop = PositionalEncoding(d), nn.Dropout(drop)
141         self.enc = nn.ModuleList(EncoderLayer(d, heads, ff, drop) for _ in range(layers))
142         self.dec = nn.ModuleList(DecoderLayer(d, heads, ff, drop) for _ in range(layers))
143         self.enc_ln, self.dec_ln = nn.LayerNorm(d), nn.LayerNorm(d)
144         for emb in (self.src_emb, self.tgt_emb): nn.init.normal_(emb.weight, mean=0.0, std=d ** -0.5)   # small embeddings: start-up
    loss is about ln(vocab)
145         self.out = nn.Linear(d, n_tgt, bias=False); self.out.weight = self.tgt_emb.weight   # tie output and embedding weights
146     def encode(self, src):                                  # runs ONCE per sentence
147         m = src_mask_of(src); x = self.drop(self.pos(self.src_emb(src) * math.sqrt(self.d)))
148         for layer in self.enc: x = layer(x, m)
149         return self.enc_ln(x), m                            # memory (B,S,d)
150     def decode(self, tgt_in, memory, src_m):                # runs once per training batch, once per output token at test time
151         y = self.drop(self.pos(self.tgt_emb(tgt_in) * math.sqrt(self.d)))
152         tm = tgt_mask_of(tgt_in)
153         for layer in self.dec: y = layer(y, memory, tm, src_m)
154         return self.out(self.dec_ln(y))                     # logits (B,T,V)
155     def forward(self, src, tgt_in):
156         memory, m = self.encode(src); return self.decode(tgt_in, memory, m)
157
158 model = Transformer(len(src_itos), len(tgt_itos))
159 print("STEP 9  parameters:", sum(p.numel() for p in model.parameters()))
160
161 # check our attention against PyTorch's built-in implementation
162 Q, K, V = (torch.randn(2, 4, 5, 16) for _ in range(3))
163 ours, _ = attention(Q, K, V); ref = F.scaled_dot_product_attention(Q, K, V)
164 print("        attention check vs torch built-in, max |difference| =", float((ours - ref).abs().max()))
165
```

Output you get (real run)

```
STEP 9  parameters: 176896
        attention check vs torch built-in, max |difference| = 2.384185791015625e-07
```

Reading the code

- **Line 135:** Defaults: width 64, 4 heads, 2 layers, feed-forward 128: small enough to train on a laptop in a minute.
- **Line 144:** Small embeddings (standard deviation 1/√d). See the box below: skipping this line caused a real bug.
- **Line 145:** Weight tying: the output scores use the same table as the input embeddings.
- **Line 146:** Source ids → embeddings × √d → position → encoder layers → final LayerNorm = memory.
- **Line 150:** Target ids → embeddings × √d → position → decoder layers (given memory) → LayerNorm → logits (B, T, vocabulary).
- **Line 162:** A correctness test: our `attention` against PyTorch's built-in `scaled_dot_product_attention` on random tensors; the printed difference is rounding noise.

A REAL BUG WE HIT WHILE WRITING THIS

The first run printed **train loss 62.1** in epoch 1, when an untrained 70-word model should start near ln 70 ≈ 4.25. Cause: `nn.Embedding` starts with standard deviation 1, and with tied weights the output scores became huge. Adding the `nn.init.normal_(…, std=d ** -0.5)` line fixed it (epoch 1 loss then printed 4.59). **Lesson: always check that the first loss is close to ln(vocabulary size).**

10. Trace the shapes through one sentence

Before training, push one sentence through and print every tensor shape. This catches most wiring mistakes in a minute. Read the output as the journey of the data: ids → embedding → encoder memory → decoder logits.

scratch_transformer.py

```
167 model.eval()
168 with torch.no_grad():
169     src, tgt_in, tgt_out = make_batch(train[:1])
170     print("STEP 10 shape trace for:", pairs[train[0]]["en"])
171     print("        src ids", tuple(src.shape), "-> embedding", tuple(model.src_emb(src).shape))
172     memory, sm = model.encode(src); print("        encoder memory", tuple(memory.shape), "| src mask", tuple(sm.shape))
173     logits = model.decode(tgt_in, memory, sm)
174     print("        decoder input", tuple(tgt_in.shape), "-> logits", tuple(logits.shape), "= (batch, target length, Kannada vocabulary)")
175     print("        one attention map: cross-attention", tuple(model.dec[0].cross_attn.last_weights.shape), "= (batch, heads, target length, source length)")
176
```

Output you get (real run)

```
STEP 10 shape trace for: Thank you very much for your help.
        src ids (1, 9) -> embedding (1, 9, 64)
        encoder memory (1, 9, 64) | src mask (1, 1, 1, 9)
        decoder input (1, 6) -> logits (1, 6, 70) = (batch, target length, Kannada vocabulary)
        one attention map: cross-attention (1, 4, 6, 9) = (batch, heads, target length, source length)
```

Reading the code

- **Line 167:** Evaluation mode switches dropout off so the trace is deterministic.
- **Line 168:** No gradients are stored: faster and lighter for inference.
- **Line 172:** The memory has the same length as the source and width *d*.
- **Line 175:** Cross-attention maps are (batch, heads, target length, source length): one row of weights over the source for every target position.

WATCH OUT

If a shape is wrong, the error appears here, long before a training run. Check (batch, heads, Tq, Tk) in particular.

11 and 12. The loss with teacher forcing, and the training loop

Because the decoder receives the correct previous words, a single forward pass produces predictions for **every** target position at once. The loss is the cross-entropy between those predictions and the shifted target, ignoring padding. Then the usual four moves: `zero_grad`, `backward`, clip the gradient, `step`. Validation loss is measured each epoch without learning, and the best checkpoint is kept (early stopping).

scratch_transformer.py

```
178 def run_epoch(idx, train_mode, opt=None, bs=8):
179     model.train(train_mode); total = n = 0
180     idx = idx[:]; random.shuffle(idx) if train_mode else None
181     for b in range(0, len(idx), bs):
182         src, tgt_in, tgt_out = make_batch(idx[b:b + bs])
183         with torch.set_grad_enabled(train_mode):
184             logits = model(src, tgt_in)                                    # all positions at once (teacher
    forcing)
185             loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), tgt_out.reshape(-1), ignore_index=PAD)
186         if train_mode:
187             opt.zero_grad(); loss.backward(); torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0); opt.step()
188         total += loss.item() * len(idx[b:b + bs]); n += len(idx[b:b + bs])
189     return total / n
190
191 opt = torch.optim.AdamW(model.parameters(), lr=2e-3)
192 print("STEP 12 training (teacher forcing):")
193 best = (1e9, None)
194 for epoch in range(1, 121):
195     tr = run_epoch(train, True, opt); va = run_epoch(val, False)
196     if va < best[0]: best = (va, {k: v.clone() for k, v in model.state_dict().items()}, epoch)
197     if epoch in (1, 5, 10, 20, 40, 80, 120): print(f"        epoch {epoch:3d}  train loss {tr:.3f}  validation loss
    {va:.3f}")
198 model.load_state_dict(best[1]); print(f"        best validation loss {best[0]:.3f} at epoch {best[2]} (checkpoint
    restored)")
199
```

Output you get (real run)

```
STEP 12 training (teacher forcing):
        epoch   1  train loss 4.586  validation loss 3.747
        epoch   5  train loss 1.963  validation loss 2.020
        epoch  10  train loss 0.804  validation loss 1.351
        epoch  20  train loss 0.181  validation loss 1.000
        epoch  40  train loss 0.017  validation loss 0.772
        epoch  80  train loss 0.005  validation loss 0.868
        epoch 120  train loss 0.002  validation loss 0.826
        best validation loss 0.721 at epoch 93 (checkpoint restored)
```

Reading the code

- **Line 184:** One call, all positions: this is teacher forcing.
- **Line 185:** Flatten (B, T, V) to (B·T, V) and targets to (B·T). `ignore_index=PAD` removes padded positions from the loss.
- **Line 187:** The four-step update: clear old gradients, backpropagate, clip, step.
- **Line 187:** Gradient clipping at 1.0 stops rare huge updates.
- **Line 183:** The same function serves training and validation.
- **Line 196:** Early stopping: remember the weights with the lowest validation loss and restore them afterwards.

WATCH OUT

Call `model.train()` for training and `model.eval()` for validation (dropout!). Measure validation loss after every epoch: the training loss keeps falling, the validation loss tells you when the model starts memorising.

13. Decode: translate a new sentence

At test time there is no target to feed. `translate` runs the encoder **once**, then loops: feed the tokens produced so far to the decoder, take the scores for the **last** position, pick the highest (greedy), append it, stop at the end token. Each turn re-runs the decoder on the longer prefix; production systems cache the keys and values of earlier tokens to avoid that recomputation.

scratch_transformer.py

```
201 @torch.no_grad()
202 def translate(sentence, max_len=24):
203     model.eval()
204     src = torch.tensor([encode_src(sentence)]); memory, sm = model.encode(src)          # encoder: once
205     out = [BOS]
206     for _ in range(max_len):                                                            # decoder: once per new token
207         logits = model.decode(torch.tensor([out]), memory, sm)
208         nxt = int(logits[0, -1].argmax())                                               # greedy: most probable token
209         if nxt == EOS: break
210         out.append(nxt)
211     return " ".join(tgt_itos[i] for i in out[1:]).replace(" ?", "?").replace(" .", ".")
212
213 print("STEP 13 greedy decoding:")
214 for i in train[:2] + test[:4]:
215     tag = "train" if i in train else "test "
216     print(f"       [{tag}] {pairs[i]['en']:38s} -> {translate(pairs[i]['en'])}   (reference: {pairs[i]['kn']})")
217
```

Output you get (real run)

```
STEP 13 greedy decoding:
       [train] Thank you very much for your help.     -> ನಿಮ್ಮ ಸಹಾಯಕ್ಕೆ ತುಂಬಾ ಧನ್ಯವಾದಗಳು.   (reference: ನಿಮ್ಮ ಸಹಾಯಕ್ಕೆ ತುಂಬಾ ಧನ್ಯವಾದಗಳು.)
       [train] Please take me to the temple.          -> ದಯವಿಟ್ಟು ನನ್ನನ್ನು ದೇವಸ್ಥಾನಕ್ಕೆ ಕರೆದುಕೊಂಡು ಹೋಗಿ.   (reference: ದಯವಿಟ್ಟು ನನ್ನನ್ನು ದೇವಸ್ಥಾನಕ್ಕೆ ಕರೆದುಕೊಂಡು ಹೋಗಿ.)
       [test ] Give me one ticket.                    -> ನನಗೆ ಒಂದು ಟಿಕೆಟ್ ಕೊಡಿ.   (reference: ನನಗೆ ಒಂದು ಟಿಕೆಟ್ ಕೊಡಿ.)
       [test ] I will go to Hassan tomorrow.          -> ನಾನು ನಾಳೆ ಹಾಸನಕ್ಕೆ ಹೋಗುತ್ತೇನೆ.   (reference: ನಾನು ನಾಳೆ ಹಾಸನಕ್ಕೆ ಹೋಗುತ್ತೇನೆ.)
       [test ] Where is the bus stand?                -> ಬಸ್ ನಿಲ್ದಾಣಕ್ಕೆ ಕರೆದುಕೊಂಡು ಹೋಗಿ.   (reference: ಬಸ್ ನಿಲ್ದಾಣ ಎಲ್ಲಿದೆ?)
       [test ] I am going to Belagavi today.          -> ನಾನು ಇಂದು ಬೆಳಗಾವಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.   (reference: ನಾನು ಇಂದು ಬೆಳಗಾವಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.)
```

Reading the code

- **Line 201:** Decorator: no gradients inside the function.
- **Line 204:** Encoder outside the loop: the source never changes, so its memory is reused every turn.
- **Line 207:** The decoder gets `<s>` plus everything chosen so far.
- **Line 208:** Only the last position matters: its scores are for the next word. `argmax` = greedy.
- **Line 209:** Stop at the end token; `max_len` is a safety limit.

WATCH OUT

Look at the output honestly: seen sentences are reproduced, most unseen ones are right, but one test sentence (*Where is the bus stand?*) comes out as a different, plausible phrase. A tiny training set cannot cover every pattern.

14. Evaluate with BLEU

BLEU counts how many 1- to 4-word sequences of the candidate also occur in the reference (clipped so no word is credited more often than it appears there), multiplies the precisions geometrically, and penalises outputs shorter than the reference. The function below is the whole metric, written from scratch.

scratch_transformer.py

```
219 def ngrams(t, n):
220     c = {}
221     for i in range(len(t) - n + 1): c[tuple(t[i:i + n])] = c.get(tuple(t[i:i + n]), 0) + 1
222     return c
223 
224 def corpus_bleu(cands, refs):
225     match, total, c_len, r_len = [0] * 4, [0] * 4, 0, 0
226     for c, r in zip(cands, refs):
227         c_len += len(c); r_len += len(r)
228         for n in range(1, 5):
229             cg, rg = ngrams(c, n), ngrams(r, n)
230             match[n - 1] += sum(min(v, rg.get(g, 0)) for g, v in cg.items()); total[n - 1] += sum(cg.values())
231     p = [match[0] / max(total[0], 1)] + [(match[i] + 1) / (total[i] + 1) for i in range(1, 4)]    # +1 smoothing for n > 1
232     bp = 1.0 if c_len >= r_len else math.exp(1 - r_len / max(c_len, 1))
233     return 100 * bp * math.exp(sum(math.log(max(x, 1e-9)) for x in p) / 4) if p[0] > 0 else 0.0
234 
235 def score(idx):
236     return corpus_bleu([tokenize(translate(pairs[i]["en"])) for i in idx], [tokenize(pairs[i]["kn"]) for i in idx])
237 print(f"STEP 14 BLEU  train {score(train):.1f} | validation {score(val):.1f} | test {score(test):.1f}   (10-sentence sets: indicative only)")
238 
```

Output you get (real run)

```
STEP 14 BLEU  train 100.0 | validation 41.5 | test 63.9   (10-sentence sets: indicative only)
```

Reading the code

- **Line 219:** Counts every n-word sequence.
- **Line 230:** Clipping: a candidate n-gram earns credit at most as many times as it occurs in the reference.
- **Line 231:** Add-one smoothing for n > 1, so one missing 4-gram does not zero the whole score on short sentences.
- **Line 232:** Brevity penalty: too-short outputs are penalised by $e^{1 - r/c}$.
- **Line 235:** Translate every sentence of a split, tokenize hypothesis and reference the same way, and score.

WATCH OUT

The train score (100) only shows memorisation; the validation and test scores are the honest ones, and with 10 sentences each they are noisy. Use several thousand sentences and a standard tool (for example sacreBLEU) for real reports.

PART B · A REAL PRETRAINED MODEL WITH HUGGING FACE

1. Load the tokenizer and the model

Two objects: the **tokenizer** (text ↔ token ids) and the **model**. `src_lang` and `tgt_lang` tell NLLB which language tags to use. `attn_implementation='eager'` selects the plain attention code, which can return its weights (the faster fused kernels cannot). `torch.set_grad_enabled(False)` turns gradients off globally: this program only runs the model.

nllb_step_by_step.py

```
7  import torch
8  from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
9
10 torch.set_grad_enabled(False)                                    # inference only: no gradients, less memory
11 NAME = "facebook/nllb-200-distilled-600M"
12 SENTENCE = "I am going to Mysuru tomorrow."
13
15 tok = AutoTokenizer.from_pretrained(NAME, src_lang="eng_Latn", tgt_lang="kan_Knda")
16 model = AutoModelForSeq2SeqLM.from_pretrained(NAME, attn_implementation="eager").eval()   # eager attention so weights
   can be returned
17 cfg = model.config
18 print("STEP 1  loaded:", NAME, "|", cfg.encoder_layers, "encoder +", cfg.decoder_layers, "decoder layers | d_model",
   cfg.d_model,
19       "| heads", cfg.encoder_attention_heads, "| vocabulary", cfg.vocab_size)
20
```

Output you get (real run)

```
STEP 1  loaded: facebook/nllb-200-distilled-600M | 12 encoder + 12 decoder layers | d_model 1024 | heads 16 | vocabulary 256206
```

Reading the code

- **Line 10:** Inference only: saves memory and time.
- **Line 15:** Downloads and caches the vocabulary of 256,206 word pieces.
- **Line 16:** Needed when you want `output_attentions=True` to work.
- **Line 17:** The configuration object lists the architecture: 12 + 12 layers, width 1024, 16 heads.

WATCH OUT

The first run downloads about 2.4 GB. If memory is short, load with `torch_dtype=torch.float16` on a GPU.

2. Tokenize: text to ids

The tokenizer cuts the sentence into pieces, adds the language tag `eng_Latn` at the start and `</s>` at the end, and returns two tensors: `input_ids` and `attention_mask` (1 = real token, 0 = padding). The leading _ marks the start of a word.

nllb_step_by_step.py

```
22  enc = tok(SENTENCE, return_tensors="pt")                        # input_ids and attention_mask
23  print("STEP 2  pieces:", tok.convert_ids_to_tokens(enc["input_ids"][0]))
24  print("        ids   :", enc["input_ids"][0].tolist(), "| attention_mask:", enc["attention_mask"][0].tolist())
25
```

Output you get (real run)

```
STEP 2  pieces: ['eng_Latn', '▁I', '▁am', '▁going', '▁to', '▁Mys', 'uru', '▁tomorrow', '.', '</s>']
        ids   : [256047, 117, 259, 17221, 202, 23059, 1027, 103005, 248075, 2] | attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Reading the code

- **Line 22:** `return_tensors='pt'` gives PyTorch tensors with a batch axis of size 1.
- **Line 23:** Turns ids back into readable pieces: *Mys* + *uru* shows a rare name split in two.

3. Run the encoder once

`model.get_encoder()` returns the encoder stack. One call produces the memory: one vector of 1024 numbers for each of the 10 source pieces, already informed by the whole sentence. With `output_attentions=True` it also returns one attention map per layer.

nllb_step_by_step.py

```
27  encoder_out = model.get_encoder()(**enc, output_attentions=True)
28  memory = encoder_out.last_hidden_state                          # (batch, source length, d_model)
29  print("STEP 3  encoder memory:", tuple(memory.shape), "| self-attention maps:", len(encoder_out.attentions), "layers,
    each", tuple(encoder_out.attentions[0].shape), "= (batch, heads, S, S)")
30
```

Output you get (real run)

```
STEP 3  encoder memory: (1, 10, 1024) | self-attention maps: 12 layers, each (1, 16, 10, 10) = (batch, heads, S, S)
```

Reading the code

- **Line 27:** `**enc` passes `input_ids` and `attention_mask` as keyword arguments.
- **Line 28:** The final encoder output: the **memory** of shape (batch, S, 1024).
- **Line 29:** A tuple with one tensor per layer: (batch, heads, S, S).

4. The decoder loop, written by hand

This is what `generate()` does internally. Start with `</s>` and the target-language tag; each turn call the decoder with the tokens so far and the encoder memory, take the scores of the **last** position through `lm_head`, softmax, pick the most probable piece, append it, and stop at `</s>`. The printout shows the three best candidates at each turn, the real probabilities behind the translation.

nllb_step_by_step.py

```
32  KAN = tok.convert_tokens_to_ids("kan_Knda")                    # forces the target language
33  start = model.config.decoder_start_token_id                    # NLLB starts decoding with </s>
34  ids = [start, KAN]
35  print("STEP 4  decoding loop (greedy):")
36  for turn in range(1, 30):
37      dec = model.get_decoder()(input_ids=torch.tensor([ids]), encoder_hidden_states=memory,
    encoder_attention_mask=enc["attention_mask"])
38      logits = model.lm_head(dec.last_hidden_state[:, -1])        # scores for the NEXT token only: (1, vocabulary)
39      probs = torch.softmax(logits.float(), dim=-1)[0]
40      p, nxt = probs.max(0)
41      ids.append(int(nxt))
42      top = torch.topk(probs, 3)
43      print(f"        turn {turn:2d}: chose {tok.convert_ids_to_tokens(int(nxt))!r:16} p={float(p):.3f}   top-3:
    {[(tok.convert_ids_to_tokens(int(i)), round(float(v), 3)) for v, i in zip(top.values, top.indices)]}")
44      if int(nxt) == tok.eos_token_id: break
45  manual = tok.decode(ids[2:], skip_special_tokens=True)
46  print("        manual loop result :", manual)
47
```

Output you get (real run)

```
STEP 4  decoding loop (greedy):
        turn  1: chose '▁ನಾ'           p=0.577   top-3: [('▁ನಾ', 0.577), ('▁ನಾನು', 0.354), ('▁ನನಗೆ', 0.003)]
        turn  2: chose 'ಳೆ'            p=0.951   top-3: [('ಳೆ', 0.951), ('ಳೆಯ', 0.03), ('ಳ', 0.007)]
        turn  3: chose '▁ನಾನು'         p=0.562   top-3: [('▁ನಾನು', 0.562), ('▁ಮ', 0.166), ('▁ಮೈ', 0.024)]
        turn  4: chose '▁ಮ'            p=0.692   top-3: [('▁ಮ', 0.692), ('▁ಮೈ', 0.172), ('▁ಮು', 0.014)]
        turn  5: chose 'ೈಸ'            p=0.966   top-3: [('ೈಸ', 0.966), ('ಿಸ', 0.01), ('ಸ', 0.006)]
        turn  6: chose 'ೂರ'            p=0.421   top-3: [('ೂರ', 0.421), ('ೂರು', 0.388), ('ೂರರ', 0.07)]
        turn  7: chose 'ರಿಗೆ'          p=0.793   top-3: [('ರಿಗೆ', 0.793), ('ರು', 0.093), ('ರಿನ', 0.017)]
        turn  8: chose '▁ಹೋಗ'          p=0.697   top-3: [('▁ಹೋಗ', 0.697), ('▁ಹೂರ', 0.043), ('▁ಹೋಗುವ', 0.043)]
        turn  9: chose 'ುತ್ತಿದ್ದೇನೆ'   p=0.466   top-3: [('ುತ್ತಿದ್ದೇನೆ', 0.466), ('ುತ್ತೇನೆ', 0.271), ('ಲ', 0.058)]
        turn 10: chose '.'              p=0.869   top-3: [('.', 0.869), ('</s>', 0.02), ('▁.', 0.006)]
        turn 11: chose '</s>'           p=0.901   top-3: [('</s>', 0.901), ('▁.', 0.0), ('▁-', 0.0)]
        manual loop result : ನಾಳೆ ನಾನು ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.
```

Reading the code

- **Line 34:** NLLB forces the output language by putting its tag as the first generated token.
- **Line 37:** `encoder_hidden_states=memory` is what the cross-attention reads; `encoder_attention_mask` hides source padding.
- **Line 38:** Only the last position's vector is converted to 256,206 scores.
- **Line 39:** Scores to probabilities; `.float()` avoids precision problems in half precision.
- **Line 44:** Stop at end of sequence.

WATCH OUT

Re-running the whole decoder at every turn is correct but slow; `generate()` uses a key/value cache so earlier positions are not recomputed.

5. The same thing with generate()

`generate()` wraps the loop, adds the cache, and offers decoding strategies. `num_beams=1` is greedy and must reproduce the manual loop exactly (the printed `True` confirms it). `num_beams=4` is beam search: four partial translations are kept at each turn and the best-scoring complete one is returned.

nllb_step_by_step.py

```
49 greedy = model.generate(**enc, forced_bos_token_id=KAN, num_beams=1, do_sample=False, max_new_tokens=40)
50 beam = model.generate(**enc, forced_bos_token_id=KAN, num_beams=4, max_new_tokens=40)
51 print("STEP 5  generate(greedy):", tok.decode(greedy[0], skip_special_tokens=True), "| identical to the manual loop:",
   tok.decode(greedy[0], skip_special_tokens=True) == manual)
52 print("        generate(beam=4) :", tok.decode(beam[0], skip_special_tokens=True))
53
```

Output you get (real run)

```
STEP 5  generate(greedy): ನಾಳೆ ನಾನು ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ. | identical to the manual loop: True
        generate(beam=4) : ನಾಳೆ ನಾನು ಮೈಸೂರಿಗೆ ಹೋಗುತ್ತಿದ್ದೇನೆ.
```

Reading the code

- **Line 49:** Same language forcing as in the manual loop.
- **Line 49:** Greedy decoding.
- **Line 50:** Beam search; the sum of log-probabilities (length-normalised) ranks the finished hypotheses.
- **Line 51:** A good habit: prove your hand-written loop equals the library.

6. Inspect the attention

Passing `output_attentions=True` to the decoder returns two sets of maps per layer: **decoder self-attention** (T×T, causal) and **cross-attention** (T×S, target looks at source). The code averages the 16 heads of the last layer and prints, for the pieces of the place name, which source piece gets the most weight and how much weight sits on the two marker tokens.

nllb_step_by_step.py

```
55  dec_in = torch.tensor([ids[:-1]])                                  # what the decoder saw: </s> kan_Knda + its own output
56  out = model.get_decoder()(input_ids=dec_in, encoder_hidden_states=memory, encoder_attention_mask=enc["attention_mask"],
    output_attentions=True)
57  print("STEP 6  decoder self-attention per layer:", tuple(out.attentions[0].shape), "| cross-attention per layer:",
    tuple(out.cross_attentions[0].shape), "= (batch, heads, T, S)")
58  cross = out.cross_attentions[-1][0].mean(0)                        # last layer, average over heads: (T, S)
59  src_pieces = tok.convert_ids_to_tokens(enc["input_ids"][0])
60  for t in (4, 5, 6):                                                # rows where the decoder writes the pieces of the place
    name
61      row = cross[t]; j = int(row[1:-1].argmax()) + 1                # ignore the language tag and </s>
62      print(f"        decoder input {tok.convert_ids_to_tokens(int(dec_in[0, t]))!r:12} -> looks most at source
    {src_pieces[j]!r} ({float(row[j]):.2f}); weight on the two markers: {float(row[0] + row[-1]):.2f}")
63
```

Output you get (real run)

```
STEP 6  decoder self-attention per layer: (1, 16, 12, 12) | cross-attention per layer: (1, 16, 12, 10) = (batch, heads, T, S)
        decoder input '▁ನಾನು'      -> looks most at source '▁Mys' (0.38); weight on the two markers: 0.51
        decoder input '▁ಮ'         -> looks most at source '▁Mys' (0.10); weight on the two markers: 0.81
        decoder input 'ೈಸ'         -> looks most at source 'uru' (0.07); weight on the two markers: 0.90
```

Reading the code

- **Line 55:** Teacher-force the produced output: feed everything except the last piece, as in training.
- **Line 58:** Last layer, first batch item, mean over heads: a (T, S) matrix of percentages.
- **Line 61:** Ignore the language tag and `</s>` when asking which word is most attended.

WATCH OUT

Averaged attention is a rough picture: weight often rests on the end marker, as the printout shows (0.51 to 0.90 on the markers). Do not over-interpret it.

PART B · A REAL PRETRAINED MODEL WITH HUGGING FACE

7. Verify one attention head by hand

The scaled dot-product maths from Part A, applied to the real model: capture the input the cross-attention module receives (a forward pre-hook), project it with the module's own `q_proj` and `k_proj`, take the 64 numbers of head 1, compute $Q \cdot K^T / \sqrt{64}$, softmax, and compare with the weights the model returned. The difference is exactly zero.

nllb_step_by_step.py

```
65  L, HEAD = 5, 0                                                  # decoder layer 6, head 1
66  layer = model.get_decoder().layers[L].encoder_attn              # the cross-attention module
67  captured = {}
68  hook = layer.register_forward_pre_hook(lambda m, a, kw: captured.update(q_in=(a[0] if a else kw["hidden_states"]),
    kv=kw["key_value_states"]), with_kwargs=True)
69  ref = model.get_decoder()(input_ids=dec_in, encoder_hidden_states=memory, encoder_attention_mask=enc["attention_mask"],
    output_attentions=True).cross_attentions[L][0, HEAD]
70  hook.remove()
71  dk = layer.head_dim
72  Q = layer.q_proj(captured["q_in"][0]).view(-1, layer.num_heads, dk)[:, HEAD]   # (T, 64)
73  K = layer.k_proj(captured["kv"][0]).view(-1, layer.num_heads, dk)[:, HEAD]     # (S, 64)
74  scores = Q @ K.T / math.sqrt(dk)                                  # Q K^T / sqrt(d_k)
75  mine = torch.softmax(scores, dim=-1)
76  print(f"STEP 7  Q {tuple(Q.shape)}, K {tuple(K.shape)} -> scores {tuple(scores.shape)} -> softmax; max |ours - model| =
    {float((mine - ref).abs().max()):.1e}")
77  print("        row for decoder input", repr(tok.convert_ids_to_tokens(int(dec_in[0, 5]))), "(sums to",
    round(float(mine[5].sum()), 3), "):", [round(float(v), 3) for v in mine[5]])
78
```

Output you get (real run)

```
STEP 7  Q (12, 64), K (10, 64) -> scores (12, 10) -> softmax; max |ours - model| = 0.0e+00
        row for decoder input '▁ಮು' (sums to 1.0 ): [0.005, 0.005, 0.009, 0.016, 0.023, 0.005, 0.015, 0.003, 0.004, 0.916]
```

Reading the code

- **Line 68:** A hook that copies the module's inputs while the model runs; `hook.remove()` unregisters it.
- **Line 72:** The model's own projection matrices applied to the captured activations.
- **Line 72:** Split 1024 numbers into 16 heads of 64 and pick head 0.
- **Line 74:** Exactly the formula of step 5 of Part A.
- **Line 76:** 0.0: the textbook formula and the library agree.

8. Batching and padding

To translate many sentences at once, pad them to the same length with `padding=True`. The `attention_mask` marks the padding with 0, and the model uses it so padded positions are ignored: this is the source mask of Part A step 3, built for you. `batch_decode` turns the generated ids back into text.

nllb_step_by_step.py

```
80  batch = tok(["Where is the bus stand?", "Thank you very much for your help."], return_tensors="pt", padding=True)
81  print("STEP 8  batch of 2 sentences padded to the same length:", tuple(batch["input_ids"].shape), "| attention_mask:",
    batch["attention_mask"].tolist())
82  res = model.generate(**batch, forced_bos_token_id=KAN, num_beams=4, max_new_tokens=40)
83  for s, r in zip(["Where is the bus stand?", "Thank you very much for your help."], tok.batch_decode(res,
    skip_special_tokens=True)): print("        ", s, "->", r)
84
```

Output you get (real run)

```
STEP 8  batch of 2 sentences padded to the same length: (2, 10) | attention_mask: [[1, 1, 1, 1, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1,
1, 1, 1, 1, 1]]
         Where is the bus stand? -> ಬಸ್ ನಿಲ್ದಾಣ ಎಲ್ಲಿದೆ?
         Thank you very much for your help. -> ನಿಮ್ಮ ಸಹಾಯಕ್ಕಾಗಿ ತುಂಬಾ ಧನ್ಯವಾದಗಳು.
```

Reading the code

- **Line 80:** Pads to the longest sentence in the batch.
- **Line 81:** The printed zeros mark the two padding slots of the first sentence.
- **Line 83:** Decodes all outputs at once; `skip_special_tokens=True` removes tags and end markers.

WATCH OUT

Always pass the attention mask (the `**batch` pattern does). Without it the padding is treated as real words and results change.

SUMMARY

Every tensor, from start to end

Stage	Variable (Part A)	Shape	Meaning
Tokenized source	`src`	(B, S)	integer ids, padded with 0
Decoder input	`tgt_in`	(B, T)	<s> y1 … y(T−1), padded
Training target	`tgt_out`	(B, T)	y1 … yT </s> (the input shifted left)
Source mask	`src_mask_of(src)`	(B, 1, 1, S)	True for real source tokens
Target mask	`tgt_mask_of(tgt_in)`	(B, 1, T, T)	lower triangle AND not padding
Embedded source	`src_emb(src)`	(B, S, d)	one vector per token (then × √d + position)
Queries, keys, values	`Q, K, V`	(B, h, L, d/h)	after the three linear layers and the head split
Attention scores / weights	`scores` , `weights`	(B, h, Tq, Tk)	weights sum to 1 over Tk
Encoder memory	`memory`	(B, S, d)	computed once; keys and values of every cross-attention
Decoder state	`y`	(B, T, d)	updated by 3 sub-layers per decoder layer
Logits	`logits`	(B, T, V)	one score per vocabulary word, per position
Loss	`loss`	scalar	mean cross-entropy over non-padding positions

Training versus translating, in code terms

	Training (teacher forcing)	Translating (the loop)
Decoder input	correct target, shifted	its own previous outputs
Calls to `encode`	once per batch	once per sentence
Calls to `decode`	once per batch (all positions)	once per new token
Dropout, gradients	`model.train()` , gradients on	`model.eval()` , `no_grad`
Result	a loss, then `backward()` and `step()`	token ids, then `decode` back to text

Debugging checklist (each item was useful while writing this guide)

First loss ≈ ln(vocabulary)	Here ln 70 ≈ 4.25 and epoch 1 gave 4.59. A value like 62 means mis-scaled initialisation (see the bug box in step 9).
Print every shape once	Step 10. Wrong head or mask shapes show up immediately.
Compare with a reference	Our attention against `F.scaled_dot_product_attention` (difference 2e-7); our decode loop against `generate()` ; our head maths against the model's weights (difference 0).
Overfit a tiny set first	If the model cannot reach near-zero training loss on 50 pairs (ours reached 0.002), something is broken.
Mask and shift sanity	Print the causal matrix (step 3). Check `tgt_in` and `tgt_out` are offset by exactly one.
train() vs eval()	Forgetting `eval()` leaves dropout on during decoding and gives noisy output.
Validation, not training, decides	Training loss fell to 0.002 while validation loss bottomed near 0.72; the best checkpoint is restored.